Guide Technique travaux pratiques - POO & TDD

Architecture Logicielle -- Java & Python

TP1 : Heritage - TP2 : Encapsulation - TP3 : Polymorphisme

Paramètre	Valeur
Module	Module 1 -- Modélisation Objet Avancée
Public	2eme année Ingénieur -- ENSTA Brest -- binômes ou trinôme
Durée totale	(TP1 + TP2 + TP3) -- Pratique guidée TDD
Philosophie	TDD strict -- RED -> GREEN -> REFACTOR -- cycle par cycle
Fil rouge	Systeme de gestion étudiante : Personne / Etudiant / Enseignant
Langages	Python 3.10+ et Java 17 LTS
Framework	pytest 7+ (Python) -- JUnit 5 (Java)
Evaluation	40 % TP livrable sur Git -- 60 % Contrôle Continu

Objectif global

Ces trois TP forment une progression complète. Le **fil rouge unique** est un systeme de gestion étudiante enrichi TP par TP : vous construisez d'abord les classes (**Heritage**), vous les sécurisez (**Encapsulation**), puis vous les rendez polymorphes (**Polymorphisme**). Chaque TP s'appuie strictement sur l'approche **TDD**.

Roadmap Technique Commune -- Fil Rouge TDD

Obligatoire -- Suivez ces étapes dans l'ordre pour les 3 TP. Chaque TP repart exactement là ou le précèdent s'est arrête.

Etape	Objectif	Python	Java
1	Environnement	`git init && python -m venv .venv`	`mvn archetype:generate (JUnit 5)`
2	Structure fichiers	`cours.py personne.py etudiant.py test_*.py`	`Cours.java Personne.java Etudiant.java TestX.java`
3	RED -- 1er test	`pytest test_cours.py -> FAILED (normal)`	`@Test -> erreur compilation (normal)`
4	class Cours	`class Cours: self.nom_cours self.professeur`	`class Cours { String nomCours; String professeur; }`
5	class Personne	`class Personne: self.nom self.age`	`class Personne { String nom; int age; }`
6	Heritage Etudiant	`class Etudiant(Personne): super().__init__(nom,age)`	`class Etudiant extends Personne { super(nom,age); }`
7	GREEN -- tests OK	`pytest -> PASSED (vert)`	`mvn test -> BUILD SUCCESS`
8	Encapsulation	`self.__age @age.setter -> ValueError`	`private int age; setAge() -> throw Exception`
9	Enseignant	`class Enseignant(Personne): self.salaire`	`class Enseignant extends Personne { double salaire; }`
10	afficher_details()	`def afficher_details(self): @Override`	`@Override void afficherDetails() dans chaque classe`
11	Polymorphisme	`liste: list[Personne] = [etudiant, enseignant]`	`List<Personne> liste = Arrays.asList(e1, ens1)`
12	REFACTOR + Git	`git add . && git commit -m 'feat(TP1/2/3)'`	`git add . && git commit -m 'feat(TP1/2/3)'`

Cycle TDD -- La règle d’or

RED Ecrire le test AVANT le code	Le fichier de test est créé **En premier.** Il échoue -- c’est normal et attendu. C’est la preuve que le test est utile.
GREEN Minimum pour PASSER	Implémenter juste assez pour que le test passe. **Pas de sur-ingénierie.** Un seul objectif : test vert.
REFACTOR Améliorer SANS casser	Nettoyer, renommer, factoriser. **Les tests restent verts.** Commit Git à chaque TP.

TP 1 -- Heritage & TDD -- Application de Gestion Etudiante

1.1 Contexte & Objectif

Vous construisez un **systeme de gestion étudiante** à partir de zéro. Le principe d'héritage permet à `Etudiant` de réutiliser le code de `Personne` et d'y ajouter ses spécificités académiques.

* **Principe vise :** Heritage -- une classe fille etend une classe mère
* **Règle Java :** super(args) doit être la première instruction du constructeur fils
* **règle Python :** super().__init__() obligatoire + respect du MRO

1.2 Architecture des classes

Classe	Détail
Cours	**Attributs :** `nom_cours` (str/String) -- `professeur_responsable` (str/String)
Personne (mère)	**Attributs :** `nom` (str/String) -- `age` (int)
Etudiant (fille)	**Herite de Personne +** `numero_etudiant` -- `moyenne` -- `liste_cours`
Methode clé	`ajouter_cours(cours)` -- `__str__` / `toString()` -- `super().__init__` / `super()`

1.3 Phase RED -- Ecrire les tests AVANT le code

règle absolue : Créez les fichiers de test **avant** toute implémentation. Ils doivent **échouer** -- sinon le test est inutile.

Python -- test_heritage.py

Java -- TestHeritage.java

1.4 Phase GREEN -- Implémenter les classes

Implémentez **une methode à la fois**. Lancez les tests après chaque ajout. Objectif : tous les tests verts.

Etape TDD	Python - pytest	Java - JUnit 5
class Cours	`class Cours:`	`public class Cours {`
Attributs Cours	`self.nom_cours self.professeur_responsable`	`private String nomCours, professeurResponsable;`
Constructeur Cours	`def __init__(self, nom_cours, professeur):`	`public Cours(String nomCours, String professeur)`
__str__ Cours	`def __str__(self) -> f-string`	`@Override public String toString()`
class Personne	`class Personne:`	`public class Personne {`
Attributs Personne	`self.nom self.age`	`protected String nom; protected int age;`
Constructeur Personne	`def __init__(self, nom, age):`	`public Personne(String nom, int age)`
Getters Personne	`self.nom self.age (proprietes publiques)`	`getNom() getAge()`
class Etudiant	`class Etudiant(Personne):`	`class Etudiant extends Personne {`
super() obligatoire	`super().__init__(nom, age) -- 1ere ligne !`	`super(nom, age); // 1ere ligne !`
Attributs Etudiant	`self.numero_etudiant self.moyenne self.liste_cours=[]`	`private String numEtudiant; private double moyenne; private List<Cours>`
ajouter_cours()	`def ajouter_cours(self, cours):`	`public void ajouterCours(Cours c)`
__str__ Etudiant	`def __str__ -> super().__str__() + cours`	`@Override toString() -> super.toString() + cours`

1.5 Main -- Programme de démonstration

1.6 Phase REFACTOR + Livrables

* Vérifier qu'instance `(alice, Personne)` / `assertInstanceOf(Personne.class, alice)` retournent **True**.
* Ajouter `__repr__` (Python) pour une représentation technique lisible.
* vérifier que les tests TP1 sont tous **100 % GREEN** avant de passer au TP2.

* **Git :** `git add . && git commit -m 'feat(TP1): Heritage Personne Etudiant Cours'`

Livrable TP1	Contenu
Code source	cours.py personne.py etudiant.py main.py / Cours.java Personne.java Etudiant.java Main.java
Tests	test_heritage.py -- tous GREEN / TestHeritage.java -- BUILD SUCCESS
Git	commit minimum avec tous les fichiers

TP 2 -- Encapsulation & Validation -- Sécurisation des Données

2.1 Contexte & Objectif

Vous repartez du code TP1. L'objectif est de **sécuriser les données** : rendre les attributs prives et ajouter une **validation** à chaque modification. Un Age négatif ou une moyenne or [0,20] doit être immédiatement refuse.

* **Principe vise :** Encapsulation -- attributs prives + getters/setters + validation
* **Python :** Double underscore `self.__age` + `@property` + `@age.setter` -> `ValueError`
* **Java :** Champs `private` + `getAge()` + `setAge()` -> `IllegalArgumentException`

2.2 Spécifications de validation

Attribut	règle métier
age (Personne)	age STRICTEMENT entre 0 et 100 -- toute autre valeur -> exception
moyenne (Etudiant)	moyenne ENTRE 0.0 et 20.0 (inclus) -- toute autre valeur -> exception
nom (Personne)	nom NE PEUT PAS etre vide ou None/null -- exception si vide
numero_etudiant	Lecture seule apres creation -- aucun setter expose
liste_cours	Accessible uniquement via ajouter_cours() -- jamais directement

2.3 Phase RED -- Tests d'encapsulation

Python -- test_encapsulation.py

Java -- TestEncapsulation.java

2.4 Phase GREEN -- Modifier les classes pour l'encapsulation

Ne reprenez Personne et `Etudiant` du TP1 et ajoutez l'encapsulation. Les tests TP1 **doivent rester verts**.

Etape TDD	Python - pytest	Java - JUnit 5
Rendre age prive	`self.__age = 0 (initialisation)`	`private int age;`
@property age	`@property def age(self) -> int:`	`public int getAge()`
@age.setter + valid.	`@age.setter if val<0 or val>100: raise ValueError`	`public void setAge(int v) { if(v<0||v>100) throw new IllegalArgumentException }`

Rendre moy privee	`self.__moyenne = 0.0`	`private double moyenne;`
@moyenne.setter+valid.	`@moyenne.setter if not 0<=val<=20: raise ValueError`	`public void setMoyenne(double m) { if(m<0||m>20) throw... }`
Validation nom	`if not nom or not nom.strip(): raise ValueError`	`if(nom==null||nom.isEmpty()) throw new IllegalArgumentException`
Constructeur Personne	`Appeler self.age = age (via setter avec validation)`	`Appeler this.setAge(age) dans le constructeur`
Constructeur Etudiant	`Appeler self.moyenne = moyenne (via setter)`	`Appeler this.setMoyenne(moyenne) dans le constructeur`

2.5 Démonstration encapsulation -- Main TP2

2.6 Phase REFACTOR + Livrables

* vérifier que les tests TP1 **restent tous verts** après ajout de l'encapsulation.
* vérifier que les attributs prives sont inaccessibles depuis l'extérieur de la classe.
* Lancer `pytest test_heritage.py test_encapsulation.py -v` : 100 % GREEN.
* **Git :** `git add . && git commit -m 'feat(TP2): encapsulation validation age moyenne'`

Livrable TP2	Contenu
Code source modifie	personne.py etudiant.py (attributs prives + setters valides) / idem Java
Tests	test_encapsulation.py + test_heritage.py -- tous GREEN
Git	commit TP2 -- 2 commits au total (TP1 + TP2)

TP 3 -- Polymorphisme & Abstraction -- Extension du Systeme

3.1 Contexte & Objectif

L'école gère maintenant des **Etudiants ET des Enseignants**. Ils sont tous des `Personne` mais leur comportement diffère. Le polymorphisme permet de les traiter **uniformément** via une liste de type `Personne`, chacun ayant sa propre version de `afficher_details()`.

* **Principe vise :** Polymorphisme -- liaison dynamique -- methodes redefinies (Override)
* **Methode cle :** `afficher_details()` definie dans `Personne`, redefinie dans `Etudiant` et `Enseignant`
* **Defi :** Une liste `list[Personne]` / `List<Personne>` contient les deux types -- appel identique, comportement different

3.2 Nouvelles classes

Classe	Detail
Personne (modifiee)	Ajouter : def afficher_details(self) / void afficherDetails() -- methode de base
Etudiant (modifie)	Redefinir afficher_details() : affiche nom, age, moyenne, liste cours
Enseignant (nouvelle)	Herite de Personne -- ajouter : salaire (float/double) -- Redefinir afficher_details() : affiche nom, age, salaire, matiere
ListePersonnes	Collection hétérogène : ne contient Personne, Etudiant ET Enseignant -- itérer et appeler afficher_details()

3.3 Phase RED -- Tests polymorphiques

Python -- test_polymorphisme.py

Java -- TestPolymorphisme.java

3.4 Phase GREEN -- Implementer le polymorphisme

Ajoutez `afficher_details()` dans chaque classe et creez `Enseignant`. Les tests TP1 et TP2 **doivent rester verts**.

Etape TDD	Python - pytest	Java - JUnit 5
Personne.afficher_details()	`def afficher_details(self) -> str: f'{nom}, {age} ans'`	`public String afficherDetails() { return nom+", "+age+" ans"; }`
Etudiant.afficher_details()	`@Override super().afficher_details() + moyenne + cours`	`@Override public String afficherDetails() { super.afficherDetails()+... }`

class Enseignant	`class Enseignant(Personne): super().__init__(nom, age)`	`class Enseignant extends Personne { super(nom, age); }`
Attributs Enseignant	`self.__matiere self.__salaire + getters + setters`	`private String matiere; private double salaire; getters/setters`
Enseignant.afficher_details()	`@Override super() + matiere + salaire`	`@Override public String afficherDetails() matiere + salaire`
Liste heterogene	`liste: list[Personne] = [etudiant, enseignant]`	`List<Personne> liste = new ArrayList<>(); liste.add(e); liste.add(ens);`
Boucle polymorphique	`for p in liste: p.afficher_details() # liaison dynamique`	`for(Personne p : liste) p.afficherDetails(); // liaison dynamique`

3.5 Programme principal -- TP3

3.6 Phase REFACTOR + Livrables

* Lancer `pytest test_heritage.py test_encapsulation.py test_polymorphisme.py -v` : **100 % GREEN**.
* vérifier qu'afficher`_details()` dans la boucle n utilise **aucun** `isinstance` / `instanceof`.
* Générer le diagramme UML (PlantUML ou Draw.io) : Personne <|-- Etudiant, Personne <|-- Enseignant.
* **Git :** `git add . && git commit -m 'feat(TP3): polymorphisme Enseignant afficher_details'`

Livrable TP3	Contenu
Code source TP3	enseignant.py personne.py etudiant.py main_tp3.py / idem Java
Tests	test_heritage.py + test_encapsulation.py + test_polymorphisme.py -- tous GREEN
Git	3 commits (TP1 + TP2 + TP3)
README	Instructions pour lancer les tests et le main

Livrables finaux -- Les 3 TP

Livrable	Contenu attendu
Structure Python	cours.py personne.py etudiant.py enseignant.py main.py main_tp2.py main_tp3.py test_heritage.py test_encapsulation.py test_polymorphisme.py
Structure Java	Cours.java Personne.java Etudiant.java Enseignant.java Main.java TestHeritage.java TestEncapsulation.java TestPolymorphisme.java pom.xml
Commits Git	feat(TP1): Heritage -- feat(TP2): Encapsulation -- feat(TP3): Polymorphisme
Tests 100 % GREEN	pytest test_heritage.py test_encapsulation.py test_polymorphisme.py -v

Critères d'évaluation

Critères	Description
TP1 Heritage (30 %)	super() appele. isinstance/assertInstanceOf valides. __str__ complet.
TP2 Encapsulation (35 %)	Attributs prives. Setters avec validation. Tests TP1 toujours verts.
TP3 Polymorphisme (35 %)	afficher_details() sans isinstance dans la boucle. Liste heterogene OK.

ANNEXE TECHNIQUE - Stack & Commandes

Stack Technique

Composant	Détail
OS	Linux Ubuntu 22.04 LTS ou Windows 10/11 + WSL2
Python	Python 3.10+
Java	JDK 17 LTS -- OpenJDK ou Oracle
IDE	Visual Studio Code -- Extension Pack for Java + Python + PlantUML
Build Java	Apache Maven 3.8+
Tests Java	JUnit 5 via Maven Surefire Plugin 3.1+
Tests Py	pytest 7+ + pytest-cov
UML	PlantUML (Graphviz) ou Draw.io (app.diagrams.net)
Git	Git 2.30+ -- GitHub ou GitLab

Commandes essentielles

Action	Commande
Creer venv Python	python -m venv .venv && source .venv/bin/activate
Installer pytest	pip install pytest pytest-cov
Lancer TP1	pytest test_heritage.py -v
Lancer les 3 TP	pytest test_heritage.py test_encapsulation.py test_polymorphisme.py -v
Coverage HTML	pytest --cov=. --cov-report=html
Tests Maven TP1	mvn test -Dtest=TestHeritage
Tous les tests Maven	mvn test
Git -- commit TP1	git add . && git commit -m 'feat(TP1): Heritage'

pom.xml -- JUnit 5

```
// Java - JUnit 5
```

```
<dependencies>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>5.10.0</version>
    <scope>test</scope>
  </dependency>
</dependencies>
<build><plugins><plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-surefire-plugin</artifactId>
  <version>3.1.2</version>
</plugin></plugins></build>
```

requirements.txt Python

```
# Python - pytest

pytest>=7.4
pytest-cov>=4.1
```